Laboratory Work 1

Number Systems, Binary Arithmetic and Signed Representation

Binary · Octal · Decimal · Hexadecimal | One's and Two's Complement

Week: 2
Duration: 2 hours 15 minutes
Format: guided theory, then individual practice
Tools: pen and paper; a C++ compiler for Section M only
Calculators: *not permitted* except where a task says so

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Explain why digital hardware stores data in base 2, and state the digits valid in any base.
- **LO2.** Convert an integer from any of the four bases (2, 8, 10, 16) into any other, using positional expansion, repeated division, and bit grouping.
- **LO3.** Add and subtract binary numbers by hand, tracking carries and borrows.
- **LO4.** Form the one's and two's complement of a fixed-width binary number.
- **LO5.** Represent signed integers in n-bit two's complement, perform signed addition and subtraction, and detect overflow.
- **LO6.** Read and write binary, octal and hexadecimal integer literals in C++, and print values in a chosen base.

Part I of this lab follows the Week 2 lecture *Number systems and conversions*. Part II extends it to binary arithmetic and signed numbers.

How this lab is organised

Work through the document in order. **Part I** explains each technique and gives a worked example; after most techniques there is a short **Quick check** that you solve immediately, before moving on. **Part II** is the graded practice set. Do not start Part II until your instructor says so.

Block	Content	Time
–	Introduction and learning outcomes	5 min
PART I — THEORY AND WORKED EXAMPLES		40 min
§1–§3	Bits, bases, positional notation	8 min
§4–§5	Any base → decimal; decimal → any base	10 min
§6–§7	Bit grouping: binary ↔ octal ↔ hex	8 min
§8	Binary addition and subtraction	6 min
§9–§12	Complements, signed arithmetic, common mistakes	8 min
–	*Short break*	5 min
PART II — PRACTICE		80 min
A–D	Conversions (table, expansion, division, grouping)	28 min
E–G	Error hunting, true/false, multiple choice	15 min
H–I	Binary and cross-base arithmetic	16 min
J–L	Complements, signed representation, overflow	16 min
M	C++ connection	5 min
N	Bonus challenges *(not timed)*	—
–	Submission checklist and wrap-up	5 min
Total		**2 h 15 min**

Notation used throughout. A subscript always names the base: $(101101)_2$, $(745)_8$, $(745)_{10}$, $(2E9)_{16}$. The string `101` on its own is ambiguous — it is 5 in binary, 65 in octal, 101 in decimal and 257 in hexadecimal. **Always write the subscript.**

PART I — Theory and worked examples

1 Why computers use base 2

A computer stores information with electronic switches. A switch has two states that hardware can tell apart quickly and reliably: current flowing, or not flowing. Ten distinguishable voltage levels would be far harder to build and far easier to corrupt by noise, so digital hardware settles on two.

Physical state	Symbol	Meaning
LOW	`0`	no current; the switch is off
HIGH	`1`	current flows; the switch is on

Bit

A **bit** (binary digit) is one such switch. It holds exactly one of two values: `0` or `1`. With n bits you can form 2^n distinct patterns, so n bits represent the unsigned range 0 through $2^n - 1$.

Octal and hexadecimal appear for human convenience, not hardware necessity. A 32-bit address written in binary is 32 characters long and easy to miscopy; the same address in hexadecimal is 8 characters. Because $8 = 2^3$ and $16 = 2^4$, both convert to and from binary by simple grouping — a property base 10 does not have.

2 A base controls which digits exist

The **base** (or **radix**) of a number system is the count of distinct digit symbols it uses. A digit must always be *smaller* than the base.

System	Base	Valid digits	Example
Binary	2	`0`, `1`	$(101101)_2$
Octal	8	`0` to `7`	$(745)_8$
Decimal	10	`0` to `9`	$(745)_{10}$
Hexadecimal	16	`0`–`9`, `A`–`F`	$(2E9)_{16}$

In hexadecimal the letters carry numeric values: `A` = 10, `B` = 11, `C` = 12, `D` = 13, `E` = 14, `F` = 15.

Invalid digit. $(128)_2$ is meaningless: binary has no digit `2` or `8`. $(2B7)_8$ is meaningless: octal has no digit `B`. Before converting anything, check that every digit is legal in the stated base.

3 Positional notation

In every base, a digit's contribution is the digit itself multiplied by a power of the base. The exponent is the digit's distance from the right, counting from 0.

$$d_{n-1}d_{n-2}\ldots d_1 d_0 \text{ (base } b) \;=\; \sum_{i=0}^{n-1} d_i \times b^{\,i}$$

The rightmost digit therefore has the *smallest* positional weight ($b^0 = 1$) and the leftmost has the largest. Useful powers to memorise:

2^i	1	2	4	8	16	32	64	128	256
8^i	1	8	64	512	4096				
16^i	1	16	256	4096	65536				

Quick check 1. Which of these are written correctly? $(481)_8$ $(1021)_2$ $(3G)_{16}$ $(3F)_{16}$ $(1101)_2$

4 Any base → decimal: positional expansion

Write each digit times its place value, then add. This one method works for every base — only the base inside the powers changes.

Worked example 4.1 — binary to decimal

Convert $(11011)_2$ to decimal.

$$(11011)_2 = 1\times2^4 + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 = 16 + 8 + 0 + 2 + 1 = (27)_{10}$$

A digit `0` contributes nothing; a digit `1` contributes its whole place value. For binary you can simply add the place values that sit above a `1`.

Worked example 4.2 — octal and hexadecimal to decimal

Octal:

$$(237)_8 = 2\times8^2 + 3\times8^1 + 7\times8^0 = 128 + 24 + 7 = (159)_{10}$$

Hexadecimal: replace each letter with its value first (B = 11).

$$(2B)_{16} = 2 \times 16^1 + 11 \times 16^0 = 32 + 11 = (43)_{10}$$

Quick check 2. Convert $(1010110)_2$ and $(1A4)_{16}$ to decimal.

5 Decimal → any base: repeated division

Divide by the target base, write down the remainder, then keep dividing the quotient until it reaches 0. **Read the remainders from bottom to top.**

Worked example 5.1 — decimal to binary

Convert $(45)_{10}$ to binary.

Division	Quotient	Remainder	
$45 \div 2$	22	1	← *last digit written*
$22 \div 2$	11	0	
$11 \div 2$	5	1	
$5 \div 2$	2	1	
$2 \div 2$	1	0	
$1 \div 2$	0	1	← *first digit written*

Reading upward: $(45)_{10} = (101101)_2$.
Verify: $32 + 8 + 4 + 1 = 45$. ✓

Worked example 5.2 — decimal to octal, decimal to hexadecimal

Octal, $(156)_{10}$**:** $156 \div 8 = 19$ r 4; $19 \div 8 = 2$ r 3; $2 \div 8 = 0$ r 2. Reading upward, $(156)_{10} = (234)_8$.
Check: $2 \times 64 + 3 \times 8 + 4 = 156$. ✓

Hexadecimal, $(702)_{10}$**:** $702 \div 16 = 43$ r 14 = E; $43 \div 16 = 2$ r 11 = B; $2 \div 16 = 0$ r 2. Reading upward, $(702)_{10} = (2BE)_{16}$.

For hexadecimal, convert any remainder from 10–15 into A–F *immediately*, while you still remember which row it came from.

Quick check 3. Convert $(100)_{10}$ to binary, to octal and to hexadecimal.

6 Binary ↔ octal: groups of three bits

Because $8 = 2^3$, one octal digit encodes exactly three bits — and the encoding never depends on the neighbouring digits. So no arithmetic is needed: you substitute.

Bits	000	001	010	011	100	101	110	111
Octal	0	1	2	3	4	5	6	7

Worked example 6.1 — both directions

Binary → octal. Split $(110101011)_2$ into threes *starting from the right*:

$$\underbrace{110}_{6}\ \underbrace{101}_{5}\ \underbrace{011}_{3} \quad\Longrightarrow\quad (653)_8$$

If the leftmost group is short, pad it with leading zeros: $(11010)_2$ becomes $(011\,010)_2 = (32)_8$.

Octal → binary. Replace each digit by its three bits, keeping every leading zero *inside* the number:

$$(653)_8: \quad 6 = 110,\ 5 = 101,\ 3 = 011 \quad\Longrightarrow\quad (110101011)_2$$

7 Binary ↔ hexadecimal: groups of four bits

Because $16 = 2^4$, the same substitution works with groups of four.

Hex	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
Hex	8	9	A	B	C	D	E	F
Binary	1000	1001	1010	1011	1100	1101	1110	1111

Worked example 7.1 — hexadecimal to binary, and the binary bridge

Hex → binary. $(2D7)_{16}$: 2 = 0010, D = 1101, 7 = 0111, so $(2D7)_{16} = (0010\,1101\,0111)_2 = (1011010111)_2$. Leading zeros may be dropped only at the far left of the finished number.

Octal → hexadecimal. There is no direct digit-for-digit rule, because 8 and 16 are different powers of two. Go through binary: expand in threes, then regroup in fours from the right.

$$(735)_8 \;\to\; \underbrace{111}_{7}\ \underbrace{011}_{3}\ \underbrace{101}_{5} = (111011101)_2 \;\to\; \underbrace{0001}_{1}\ \underbrace{1101}_{D}\ \underbrace{1101}_{D} = (1DD)_{16}$$

Hexadecimal → octal. The same bridge in reverse: expand in fours, regroup in threes.

$$(3A7)_{16} \;\to\; \underbrace{0011}_{3}\ \underbrace{1010}_{A}\ \underbrace{0111}_{7} = (001110100111)_2 \;\to\; \underbrace{001}_{1}\ \underbrace{110}_{6}\ \underbrace{100}_{4}\ \underbrace{111}_{7} = (1647)_8$$

Group from the right, always. Grouping $(1011010111)_2$ from the *left* gives `101 101 011 1`, which is wrong. The rightmost bit must stay in the rightmost group, because that is where the place values are anchored.

Quick check 4. Convert $(526)_8$ to hexadecimal, going through binary.

8 Binary addition and subtraction

Addition rules

$0 + 0$	$= 0$
$0 + 1 = 1 + 0$	$= 1$
$1 + 1$	$= 0$, carry 1
$1 + 1 + 1$	$= 1$, carry 1

Subtraction rules

$0 - 0 = 1 - 1$	$= 0$
$1 - 0$	$= 1$
$0 - 1$	$= 1$, borrow 1

Work column by column from the right, exactly as in decimal long addition.

Worked example 8.1 — addition and subtraction

(a) $(10101)_2 + (1101)_2$

carry	11101	
	10101	
+	01101	
	100010	

Check: $21 + 13 = 34 = (100010)_2$. ✓

(b) $(11100)_2 - (1011)_2$

borrow	11	
	11100	
−	01011	
	10001	

Check: $28 - 11 = 17 = (10001)_2$. ✓

A carry is written above the column it will be *added into*; a borrow is written above the column it is *taken from*. In (b) the rightmost column needs $0 - 1$, so it borrows from its left neighbour, which is itself 0 and must borrow in turn.

Arithmetic in octal or hexadecimal. The most reliable hand method is to convert both operands to binary, add or subtract there, then convert the result back to the original base. For example $(2F)_{16} + (1C)_{16}$: $(101111)_2 + (11100)_2 = (1001011)_2 = (75)_{10} = (4B)_{16}$.

9 One's complement

Fix a width first — 6 bits, 8 bits, whatever the problem states — and pad the number to that width. The **one's complement** is then obtained by flipping every bit: each 0 becomes 1 and each 1 becomes 0.

Worked example 9.1

8-bit number:	0001 0100
one's complement:	1110 1011

Every column changed; nothing was carried or borrowed.

10 Two's complement

The **two's complement** is the one's complement *plus one*, computed at the fixed width and discarding any carry that falls off the left end.

Worked example 10.1 — two ways to get the same answer

Take the same 8-bit number 0001 0100 (decimal 20).

Method 1 — flip, then add one:

$$0001\,0100 \xrightarrow{\text{flip}} 1110\,1011 \xrightarrow{+1} 1110\,1100$$

Method 2 — copy-and-flip (faster by hand): scanning from the right, copy every 0 and the first 1 you meet, then flip everything to the left of it.

$$0001\,0100 : \text{ copy } 00,\ \text{copy the first 1},\ \text{flip the rest } \Rightarrow\ 1110\,1100$$

Check: 1110 1100 = 236 and 236 + 20 = 256 = 2^8. That is the defining property: the two's complement of x at width n is $2^n - x$.

Quick check 5. Give the one's and two's complement of the 6-bit number $(010011)_2$.

11 Signed numbers in two's complement

Two's complement is how essentially every modern processor — and every C++ `int` — stores negative integers. One rule governs everything:

n-bit two's complement

- The leftmost bit is the **sign bit**: `0` means the value is zero or positive, `1` means negative.
- To encode a negative value $-x$: write x in n bits, then take its two's complement.
- To decode a pattern whose sign bit is `1`: take its two's complement to recover the magnitude, and put a minus sign in front.
- The representable range is -2^{n-1} through $+2^{n-1} - 1$. Zero has exactly one encoding.

Width	Most negative	Most positive	Distinct values
6 bits	−32	+31	64
8 bits	−128	+127	256
16 bits	−32768	+32767	65536

The range is asymmetric — one extra negative value — because the single pattern for zero frees up a slot on the negative side. In 6 bits, $(\texttt{100000})_2$ is −32, and +32 simply cannot be written.

Subtraction becomes addition

The reason hardware likes this representation: $A - B$ is computed as A + (two's complement of B). A processor needs only an adder. Any carry out of the leftmost column is discarded.

Worked example 11.1 — 14 + (−9) in 6-bit two's complement

1. $+14 = (\texttt{001110})_2$.
2. $9 = (\texttt{001001})_2$; flip → `110110`; add one → $-9 = (\texttt{110111})_2$.
3. Add the two patterns:
$$\texttt{001110} + \texttt{110111} = \texttt{1}\,\texttt{000101}$$
4. Discard the carry that left the 6-bit field: the answer is $(\texttt{000101})_2$.
5. Sign bit is `0`, so the value is positive: $(\texttt{5})_{10}$. And indeed $14 - 9 = 5$. ✓

Worked example 11.2 — a negative result: −12 + 7

-12: $12 = \texttt{001100} \rightarrow \texttt{110011} \rightarrow (\texttt{110100})_2$. $\quad +7 = (\texttt{000111})_2$.

$$\texttt{110100} + \texttt{000111} = \texttt{111011} \qquad \text{(no carry out)}$$

The sign bit is `1`, so the result is negative. Take the two's complement of $(\texttt{111011})_2$ to read the magnitude: $\texttt{000100} + 1 = \texttt{000101} = 5$. The answer is $(\texttt{-5})_{10}$, which matches $-12 + 7$. ✓

Overflow

Overflow rule

Overflow means the true result does not fit in n bits. For two's complement addition:

> *If the two operands have the **same** sign and the result has the **opposite** sign, the answer is wrong.*

If the operands have different signs, overflow is impossible. **A carry out of the leftmost column is not, by itself, overflow** — Worked example 11.1 produced a carry and a perfectly correct answer.

Worked example 11.3 — overflow in 6 bits

Compute 25 + 12 in 6-bit two's complement.

$$011001 + 001100 = 100101$$

Both operands were positive (sign bits 0), but the result's sign bit is 1, which decodes as −27. **Overflow.** The true answer 37 is outside the 6-bit range $[-32, +31]$, so no 6-bit pattern could have been correct. Widening to 7 or 8 bits fixes it: in 8 bits, 00011001 + 00001100 = 00100101 = 37. ✓

Quick check 6. Compute −15 − 8 in 6-bit two's complement. Is there overflow?

12 Common mistakes — and how to catch them

Mistake	What to do instead
Wrong direction	Repeated-division remainders are read **bottom to top**. The *last* remainder you computed is the *leftmost* digit.
Wrong grouping	Form groups of 3 (octal) or 4 (hex) **starting from the right**, padding the leftmost group with zeros.
Invalid digit	Every digit must be smaller than the base. An 8 can never appear in octal; a 2 can never appear in binary.
Letter values	A = 10 . . . F = 15. C is 12, not 13.
Missing subscript	Write $(101)_2$ or $(101)_{10}$. Without the subscript the value is undefined.
Forgetting the +1	One's complement is flip only; two's complement is flip **and add one**.
Ignoring the width	Complements are meaningless without a stated bit width. Pad to the width first.
Confusing carry with overflow	Check the *signs*, not the carry-out.

The habit to build: before you accept any answer, **convert it back by a different method**. Converting $(45)_{10}$ to $(101101)_2$ by division, then verifying 32 + 8 + 4 + 1 = 45 by expansion, catches almost every arithmetic slip in this lab.

PART II — Practice

Rules for Part II. Show your working, not only the final answer — a correct answer with no method earns half credit. Write the base subscript on every number you write down. No calculators except in Section M. Total: **100 points** plus up to 10 bonus points in Section N.

A Warm-up: the reference table

[3 min] (6 points)

Complete the missing cells. You will use this table constantly for the rest of the lab.

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0	0	0	10	1010	12	A
1	1	1	1	11			B
2	10	2	2	12	1100	14	
3	11	3	3	13		15	D
4	100	4	4	14	1110		E
5		5	5	15	1111	17	
6	110		6	16		20	10
7	111	7		17	10001		11
8		10	8	18	10010	22	
9	1001		9	19		23	13

Observe: octal rolls over from `7` to `10` at decimal 8, and hexadecimal rolls over from `F` to `10` at decimal 16. In every base, `10` means "one whole base".

B Positional expansion: convert to decimal

[8 min] (10 points)

Write out the expansion, then the sum. Example of the expected form: $(1101)_2 = 8 + 4 + 0 + 1 = (13)_{10}$.

B.1 Binary → decimal

a) $(101101)_2$

b) $(1110101)_2$

c) $(11100010)_2$

d) $(10110101)_2$

B.2 Octal → decimal

a) $(27)_8$

b) $(376)_8$

c) $(457)_8$

d) $(731)_8$

B.3 Hexadecimal → decimal

a) $(2D)_{16}$

b) $(1F4)_{16}$

c) $(2E5)_{16}$

d) $(4A1)_{16}$

C Repeated division: convert from decimal

[9 min] (10 points)

Show the division chain (quotient and remainder on each line), then read the remainders upward.

C.1 Decimal → binary

a) $(37)_{10}$

b) $(117)_{10}$

c) $(300)_{10}$

d) $(418)_{10}$

C.2 Decimal → octal

a) $(57)_{10}$

b) $(296)_{10}$

c) $(345)_{10}$

d) $(700)_{10}$

C.3 Decimal → hexadecimal

a) $(126)_{10}$

b) $(432)_{10}$

c) $(1023)_{10}$

d) $(2046)_{10}$

D Bit grouping

[8 min] (12 points)

D.1 Convert each hexadecimal number to binary, octal and decimal

Use four-bit groups to reach binary, then regroup in threes to reach octal.

a) $(6C)_{16}$ binary: octal: decimal:

b) $(B3E)_{16}$ binary: octal: decimal:

c) $(\texttt{7D})_{16}$ binary: octal: decimal:

d) $(\texttt{C4A})_{16}$ binary: octal: decimal:

D.2 Cross-convert through the binary bridge

e) $(\texttt{526})_8 \rightarrow$ hexadecimal

f) $(\texttt{9F2})_{16} \rightarrow$ octal

E Find the mistake

[5 min] (8 points)

Each solution below reaches a *wrong* answer. For each one, name the mistake and give the correct result.

a) A student converts $(\texttt{37})_{10}$ to binary:

```
37/2=18 r1   18/2=9 r0   9/2=4 r1   4/2=2 r0   2/2=1 r0   1/2=0 r1
```

"Reading the remainders down the page: $(\texttt{37})_{10} = (\texttt{101001})_2$*."*

b) A student converts $(\texttt{1011010111})_2$ to octal:
"Group from the left: `101 101 011 1`*, so the answer is* $(\texttt{5531})_8$*."*

c) A student evaluates $(\texttt{2B7})_8$:
"$2 \times 64 + 11 \times 8 + 7 = 128 + 88 + 7 = (\texttt{223})_{10}$."

d) A student writes -13 in 6-bit two's complement:
"$13 = (\texttt{001101})_2$*; flip the bits to get* $(\texttt{110010})_2$*; so* $-13 = (\texttt{110010})_2$."

F True or false

[5 min] (7 points)

Mark each statement **T** or **F**. For every statement you mark **F**, write one short line saying why.

1. Binary uses only the digits `0` and `1`.

2. Octal numbers use the digits `0` through `7`.

3. The rightmost digit has the smallest positional weight.

4. The leftmost digit always has the smallest positional weight.

5. Every octal digit corresponds to exactly three binary bits.

6. Every hexadecimal digit corresponds to exactly four binary bits.

7. The hexadecimal digit `C` represents decimal 13.

8. The hexadecimal digit `A` represents decimal 10.

9. Repeated division can convert a decimal integer into any other base. ____

10. Binary-to-octal conversion can be done by grouping bits in threes. ____

11. $(1111)_2$ is equal to $(\texttt{F})_{16}$. ____

12. $(10000)_2$ is equal to $(10)_{16}$. ____

13. The two's complement of a number is its one's complement plus one. ____

14. In n-bit two's complement there are two different bit patterns for zero. ____

15. With 6 bits, two's complement represents -32 through $+31$. ____

16. A carry out of the most significant bit always means overflow in two's complement. ____

17. Octal exists because 8 is a power of 2, so grouping bits is exact. ____

G Multiple choice

[5 min] (8 points)

Circle one answer per question.

1. What is the largest valid digit in base 8?
(a) 8 (b) 7 (c) 6 (d) 9

2. What is the base of the hexadecimal system?
(a) 2 (b) 8 (c) 10 (d) 16

3. How many bits represent one hexadecimal digit?
(a) 2 (b) 3 (c) 4 (d) 8

4. How many bits represent one octal digit?
(a) 2 (b) 3 (c) 4 (d) 8

5. Convert $(31)_{10}$ to hexadecimal.
(a) 1E (b) 1F (c) 2F (d) F1

6. Convert $(45)_{10}$ to hexadecimal.
(a) 2D (b) 2C (c) 3D (d) D2

7. Convert $(1010)_2$ to decimal.
(a) 8 (b) 9 (c) 10 (d) 12

8. Convert $(1101)_2$ to decimal.
(a) 11 (b) 12 (c) 13 (d) 14

9. Which binary group equals octal 6?
(a) 101 (b) 100 (c) 110 (d) 111

10. Which binary group equals octal 5?
(a) 101 (b) 100 (c) 110 (d) 111

11. What comes after `F` in hexadecimal?
(a) G (b) 10 (c) F0 (d) 11

12. What comes before `10` in hexadecimal?
(a) 9 (b) E (c) F0 (d) F

13. The 6-bit two's complement of $(001101)_2$ is
(a) 110010 (b) 110011 (c) 110100 (d) 001101

14. The range of 8-bit two's complement is
(a) 0..255 (b) $-127..127$
(c) $-128..127$ (d) $-128..128$

15. Which C++ literal equals decimal 255?
(a) `0b255` (b) `077` (c) `0xFF` (d) `0x255`

H Binary addition and subtraction

[8 min] (8 points)

Work in binary, column by column. Then verify each answer in decimal.

a) $(10101)_2 + (1101)_2$

b) $(11100)_2 - (1011)_2$

c) $(100111)_2 + (11101)_2$

d) $(110010)_2 - (10111)_2$

e) $(11010)_2 + (1011)_2$

f) $(101101)_2 - (1110)_2$

g) $(111001)_2 + (10011)_2$

h) $(100000)_2 - (1101)_2$

I Arithmetic across bases

[8 min] (9 points)

For each expression: convert both operands to binary, do the arithmetic in binary, then convert the result *back to the original base*. State the decimal check.

a) $(47 + 25)_{10}$

b) $(35 + 46)_{8}$

c) $(\texttt{2F} + \texttt{1C})_{16}$

d) $(384 - 157)_{10}$

e) $(46 + 27)_{8}$

f) $(\texttt{3A} + \texttt{2D})_{16}$

J One's and two's complement

[5 min] (5 points)

Each number below is already 6 bits wide. Give its one's complement and its two's complement. Treat the patterns as plain bit strings here — no sign is implied.

	6-bit number	One's complement	Two's complement
a)	$(\texttt{010011})_2$		
b)	$(\texttt{101100})_2$		
c)	$(\texttt{001111})_2$		
d)	$(\texttt{110010})_2$		
e)	$(\texttt{001101})_2$		
f)	$(\texttt{100011})_2$		

g) Verify one of your answers by adding the original to its two's complement. What do you get, and why?

K Signed representation

[5 min] (5 points)

Write each decimal value as a **6-bit two's complement** pattern.

a) −13

b) +22

c) −27

d) −6

e) −18

f) +25

g) What is the full range of values representable in 6-bit two's complement? Which bit pattern is the most negative value, and why has it no positive counterpart?

L Signed arithmetic and overflow

[6 min] (8 points)

Convert every operand to **6-bit two's complement**, rewrite subtraction as addition, add the patterns, discard any carry out of the 6-bit field, then decode the result. Mark overflow in the last column.

	Expression	6-bit result	Decimal	Overflow?
a)	$14 + (-9)$			
b)	$-12 + 7$			
c)	$19 - 24$			
d)	$-15 - 8$			
e)	$17 + (-11)$			
f)	$21 - 26$			
g)	$25 + 12$			
h)	$-20 + (-15)$			

i) Items **a)** and **d)** both produced a carry out of the 6-bit field, yet neither overflowed, while **g)** overflowed with no carry out. In one sentence, state the correct test for overflow.

M The C++ connection

[5 min] (4 points)

Everything above is how C++ actually stores integers. Read the program, then answer without running it.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 0b101101; // binary literal (since C++14)
    int b = 055; // octal literal (a leading 0)
    int c = 0x2D; // hexadecimal literal
    int d = 45; // decimal literal
    cout << a << " " << b << " " << c << " " << d << endl; // line 1

    cout << hex << 255 << " " << oct << 255 << " " << dec << 255 << endl; // line 2

    unsigned char u = 200;
    u = u + 100; // 300 does not fit
    cout << static_cast<int>(u) << endl; // line 3
    return 0;
}
```

a) What does **line 1** print?

b) What does **line 2** print?

c) What does **line 3** print? Explain it in binary.

d) Why does the literal `0b255` fail to compile, and what value does `077` have?

N Bonus challenges (up to 10 points, not timed)

Attempt these only after everything above is finished.

1. (2) Find the base b such that $(24)_b = (\texttt{20})_{10}$.

2. (2) Find the base b such that $(144)_b = (\texttt{100})_{10}$.

3. (2) A web colour is written `#2E8B57`. The three byte-pairs are red, green and blue. Give each as a decimal value from 0 to 255.

4. (2) An IPv4 address is four 8-bit numbers. Write `192.168.10.1` as 32 bits.

5. (1) Write -45 in **8-bit** two's complement.

6. (1) A sensor reports one of 1000 distinct readings. What is the smallest number of bits that can encode a reading, and how many patterns go unused?

Submission checklist [5 min]

Before you hand in, confirm each line. Tick the box.

- ☐ Your **name** and **group** are on the front page.
- ☐ Every number you wrote carries a **base subscript**.
- ☐ Sections A–D show the **working**, not just the answers.
- ☐ Every conversion has been **checked back** by a second method.
- ☐ Every complement in Sections J–L is exactly **6 bits** wide, leading zeros included.
- ☐ The overflow column in Section L is filled in for all eight rows.
- ☐ Sections E and M include the short **written explanations** they ask for.

Section	Available	Earned
A Reference table	6	
B Positional expansion	10	
C Repeated division	10	
D Bit grouping	12	
E Find the mistake	8	
F True or false	7	
G Multiple choice	8	
H Binary addition and subtraction	8	
I Arithmetic across bases	9	
J One's and two's complement	5	
K Signed representation	5	
L Signed arithmetic and overflow	8	
M The C++ connection	4	
Total	**100**	
N Bonus challenges	+10	

What comes next

The four ideas from this lab reappear throughout CS102:

- **Fixed width** explains why `int` overflows and why `int` and `long long` differ.
- **Two's complement** is why `-1` printed as unsigned shows as `4294967295`.
- **Hexadecimal** is how memory addresses, pointer values and colour codes are written.
- **Bit patterns** underpin bitwise operators, flags and masks later in the course.

End of Lab 1